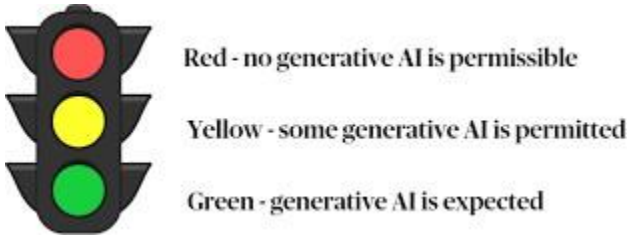
Traffic Light AI Usage Framework

This coursework uses a Traffic Light AI Usage Framework to guide how you may or may not use generative AI tools (e.g., ChatGPT, Copilot, etc.). The colour assigned to this task indicates the level of permitted AI assistance. You are expected to follow these guidelines carefully and ensure all submitted work reflects your own understanding and academic integrity.



Please check the assigned colour for this coursework and adhere to the corresponding requirements outlined below.

Red - Prohibit to use use gen AI You must not use any generative AI tools at any stage of this assignment. All work must be entirely your own. Any use of AI assistance will be considered a breach of academic integrity.
Yellow - Restricted usage of gen AI Limited use of generative AI is allowed for specific purposes only (e.g., brainstorming ideas, clarifying concepts, or language refinement). AI must not be used to generate substantive content for submission. You must clearly acknowledge any AI use according to the course guidelines.
Green - Permitted to use gen AI You may use generative AI tools to support your work, including idea generation, drafting, and editing. However, you remain responsible for the accuracy, originality, and integrity of your submission. Proper disclosure of how AI was used is required.

AI Usage Level for This Coursework: ● Yellow (Restricted Use)

Guidelines for the Responsible Use of Generative AI (GenAI)

You may use generative AI (GenAI) tools to support learning and development in this assignment in the following ways:

- **Concept Clarification:** Using GenAI to explain general concepts related to data structures and algorithms, such as stacks, queues, circular queues, priority queues, trees/BSTs, searching, traversal, insertion, deletion, and other relevant concepts.
- **Syntax & Toy Examples:** Generating isolated, small C++ code snippets to learn C++ syntax, basic programming constructs, or generic library usage. The examples **must not implement or be directly related to the specific functionality, data structures, algorithms, or problem logic required in this assignment**.
- **Debugging Assistance:** Using GenAI to identify syntax errors, compiler errors, exceptions, or generic runtime errors in code that you have written yourself. You must understand and verify any suggested corrections.
- **Brainstorming & Design Discussion:** Using GenAI to discuss high-level ideas, clarify requirements, or identify possible edge cases that may need to be considered when designing your system. However, the final data structure selection, algorithm design, implementation approach, and design decisions must be your own.
- **Learning Support:** Using GenAI to explain programming concepts, terminology, or error messages that you do not understand.

The following uses of GenAI are NOT permitted:

- **Complete or Partial Solution Generation:** GenAI must not be used to generate the complete assignment solution or substantial portions of the C++ implementation.
- **Core Data Structure Implementation:** You must not use GenAI to design or generate the implementation of the required data structures, including the core operations of stacks, queues, circular queues, priority queues, or the optional tree/BST.
- **Core Algorithm & Program Logic Generation:** You must not use GenAI to design, implement, or write the core algorithms, processing logic, workflows, or decision-making logic required by your assigned functionality.

- **Data Structure Selection & Justification:** You must make your own decisions about which data structure is appropriate for each task. GenAI must not be used to provide the final data structure selection or the justification that you submit as your own.
- **Automated Code Tracing:** GenAI must not be used to generate step-by-step code traces, dry runs, state changes, or execution tables required for your assessment.
- **Complexity & Performance Analysis:** GenAI must not be used to generate the final Big-O time or space complexity analysis and justification for your own implementation.
- **F2F Q&A Preparation as a Substitute for Understanding:** Students must not use GenAI to generate memorised answers for the individual F2F Q&A. During the Q&A, students are expected to demonstrate their own understanding of their code, data structures, algorithms, design decisions, and individual contribution.
- **Individual Contribution:** GenAI must not be used to create work that the student claims as their own individual contribution.

ASSIGNMENT 2 - LAB EVALUATION WORK #2

SMART CAMPUS PARCEL COLLECTION AND DELIVERY MANAGEMENT SYSTEM

Assessment	Group + Individual Evaluation
Working Duration	Week 10 - Week 14
Programming Language	C++
Total Marks	30 marks
Learning Outcome	CLO3: Demonstrate a solid understanding of how to write solutions using various data structures and algorithms. (P5, PLO3)

1. Background

APU is introducing a Smart Campus Parcel Collection and Delivery Management System to manage the increasing number of parcels received for students and staff.

The campus receives parcels from different delivery companies every day. Parcels may need to be received at the campus delivery centre, registered into the system, sorted for collection or delivery, assigned to collection counters or delivery personnel, collected by students or staff, and returned to the sender when necessary.

During peak periods, hundreds of parcels may be processed within a short period. The system must therefore provide an efficient way to manage parcels and ensure that important operational information is maintained accurately.

The system should support different operational situations, including managing parcels waiting to be processed, managing parcels requiring different levels of attention, allocating collection and delivery resources efficiently, recording recent staff activities, locating parcel information, managing parcel records, and handling exceptional situations such as missing parcels, unavailable resources, incorrect collection information and cancelled requests. The university requires an integrated system that applies appropriate data structures and algorithms to manage these operations efficiently.

Your team is required to develop a C++ prototype for the **Smart Campus Parcel Collection and Delivery Management System**.

2. Data Structure Requirements

Your team must analyse the requirements of each task and determine the most appropriate data structure and algorithm. Each component of the system involves specific tasks that can be optimized using suitable data structures such as **Stack, Queue, Circular Queue, or Tree structures**.

Consider:

- How data is stored and organized.
- How data is inserted, accessed, searched, removed and updated.
- The required processing sequence.
- Priority or urgency.
- Repeated or cyclic processing.
- Recent activity and reversal requirements.
- Time and space efficiency.
- Empty, full, invalid and boundary conditions.

Important: The assignment does not tell you which data structure must be used for a particular task. Each data structure must be meaningfully integrated into the completed system. Simply including a data structure and demonstrating basic operations without using it to solve a system requirement is not sufficient.

3. System Tasks

The following **FIVE TASKS** are **compulsory** depending on your grouping numbers and must operate as one integrated C++ prototype.

- Task 1 - Parcel Intake, Sorting and Processing Module
- Task 2 - Parcel Service and Collection Management Module
- Task 3 - Collection Counter and Delivery Resource Management Module
- Task 4 - Parcel Activity and Record Management Module
- Task 5 - Parcel Category Organization Module

Task 1 - Parcel Intake, Sorting and Processing Module

This module manages parcels when they arrive at the campus delivery centre. It is responsible for registering parcels, maintaining their information, sorting them according to their next required action, and managing parcels waiting to be processed.

Functional Requirements

1. Register a New Parcel

Allow staff to register a parcel on arrival. Assign a unique Parcel ID, record recipient and delivery information, parcel type and arrival date/time, assign an initial status, and prevent incomplete or duplicate registration.

2. Maintain and Locate Parcel Records

Maintain registered parcel records and allow staff to display, search for and view selected parcels, including relevant details and current status.

3. Sort and Process Incoming Parcels

Classify and process incoming parcels according to their next required action. A parcel may be prepared for recipient collection, assigned for campus delivery, sent for special handling, processed for return to sender, or placed on hold for investigation. Update status and route the parcel to the appropriate next stage.

4. Update or Remove Parcel Records

Allow appropriate parcel information to be updated when circumstances change, including status changes, collection, delivery, return or cancellation. Invalid operations must be prevented.

5. Handle Exceptional Parcel Conditions

Handle duplicate Parcel IDs, invalid recipient information, missing parcel information, a parcel reported as missing, searching for a non-existent parcel, processing an already completed parcel, and attempting to process an empty collection.

Task 2 - Parcel Service and Collection Management Module

After a parcel has been received and registered, it may require different services before the parcel process is completed. Examples include recipient collection, campus delivery, special handling, re-delivery and return to sender. This module manages service requests from creation until completion, cancellation or return.

Functional Requirements

1. Create a Service Request

Allow authorised staff to create a service request for a registered parcel. Assign a unique Request ID, associate it with an existing Parcel ID, record recipient and service information, assign an initial status, and prevent invalid or duplicate requests.

2. Maintain Active Service Requests

Maintain active service requests and allow staff to display requests awaiting action. Requests should be added, viewed, processed and removed when completed or cancelled.

3. Determine the Next Request to be Handled

Determine which active service request should be processed next according to processing conditions defined by the team. The team may consider service type, urgency, request time, recipient requirements or special handling and must justify its rule.

4. Process and Update Service Requests

Process a selected request and update its status. Record completion, cancellation or return information, update the associated parcel where appropriate, and prevent completed or cancelled requests from being processed again.

5. Handle Exceptional Service Conditions

Handle an empty set of active requests, invalid Request ID or Parcel ID, cancelled requests, incorrect recipient or collection information, equivalent processing conditions, attempts to process a completed request, and invalid service operations.

Task 3 - Collection Counter and Delivery Resource Management Module

This module manages collection counters and delivery resources. It must manage resource availability and allocate parcels or service requests to appropriate resources while maintaining accurate resource information.

Functional Requirements

1. Register and Maintain Collection and Delivery Resources

Allow staff to register and maintain collection counters and delivery personnel. Each resource should have a unique identifier, type and current status.

2. Display Current Resource Availability

Provide an up-to-date view of available, occupied and unavailable resources, including the parcel or request currently assigned where applicable.

3. Assign a Parcel or Service Request

Assign a parcel or service request to an appropriate available collection counter or delivery resource according to allocation rules defined by the team. Update the assignment and resource status.

4. Update Resource Status

Allow a resource to be marked occupied, available, unavailable or otherwise changed. Associated assignment information must remain consistent.

5. Handle Resource Allocation Exceptions

Handle all resources being occupied, no suitable resource being available, a resource becoming unavailable during processing, invalid Resource ID, selection of an unavailable resource, duplicate active assignment, or invalid release/update operation.

Task 4 - Parcel Activity and Record Management Module

This module maintains a history of important activities performed during parcel processing. It should allow staff to review what happened to a parcel and, where permitted, reverse an appropriate recent operation.

Functional Requirements

1. Record Parcel-Related Activities

Record important activities performed during parcel processing, with sufficient information to identify the parcel, activity and relevant time or sequence.

2. Display Activity History

Allow staff to display recent activities and the activity history for a selected parcel, including the order in which activities occurred.

3. Retrieve and Review Previous Activities

Allow staff to select a parcel and review previous activities, including processing sequence, previous status, most recent activity and relevant request or resource information.

4. Reverse or Undo an Appropriate Recent Activity

Provide a mechanism for reversing an appropriate recent activity where permitted. Identify the activity, confirm eligibility, restore relevant information and update related parcel/request/resource status where necessary.

5. Handle Activity-History Exceptions

Handle no activity history, invalid Parcel ID, an activity that cannot be reversed, an activity already reversed, invalid activity selection, or an operation that would result in an invalid parcel status.

Task 5 - Parcel Category Organisation Module

This module manages the organization and retrieval of parcel information. It allows staff to organize parcel records according to a selected key and efficiently insert, search, display, update and remove parcel information.

Functional Requirements

1. Create and Maintain the Parcel Organization Structure

Allow staff to create and maintain the tree-based organization structure for parcel records. Each parcel record should contain sufficient information to identify the parcel and support the selected organization key.

2. Insert a New Parcel Record

Allow staff to insert a new parcel record into the selected data structure. The system should validate the parcel information and prevent invalid or duplicate records where the selected key requires uniqueness.

3. Search for a Parcel or Category

Allow staff to search for a specified parcel record or organization key. The system should display the relevant parcel information when the record is found and provide an appropriate message when the requested record does not exist.

4. Display Parcel Records

Allow staff to display the stored parcel information.

5. Update or Remove a Parcel Record

Allow authorized staff to update or remove a selected parcel record. The operation must maintain the properties and structure of the selected tree implementation.

4. Integrated System Requirement

The team must develop **ONE** integrated C++ prototype. The **five compulsory** modules must exchange and update relevant information rather than operating as five unrelated programs.

1. Parcel arrives at the delivery centre.
2. Parcel is registered and its information is maintained.
3. Parcel is sorted/routed for its next action.
4. A service request is created where required.
5. An appropriate service/resource is selected.
6. A collection counter or delivery resource is assigned.
7. The parcel/service is processed.
8. Parcel and request status are updated.
9. The activity is recorded in system history.
10. The parcel is completed, collected, delivered, returned, cancelled or placed on hold/missing as appropriate.

5. C++ Implementation Requirements

- The system must be implemented using C++.
- Students must implement the required data structures themselves. The following STL containers are NOT allowed as substitutes for the required structures:
  - vector
  - list
  - stack
  - queue
  - priority queue
  - deque
- Other built-in container classes should not be used to bypass the learning requirements.
- Code should demonstrate meaningful identifiers, modular programming, appropriate comments, clear class/function organisation, validation/error handling, correctness and reasonable efficiency.

6. Group and Individual Requirements

- Each team should consist of **FOUR (4)** to **FIVE (5)** members, subject to the approved grouping arrangements.
  - **Minimum 3 members:** Each member will be responsible for one of the mandatory tasks:
    - Task 1 - Parcel Intake, Sorting and Processing Module
    - Task 2 - Parcel Service and Collection Management Module

        - Task 3 - Collection Counter and Delivery Resource Management Module
    - **Optional additional 2 members**: If the team has **four** or **five** members, the additional members can handle the optional task(s):
        - Task 4 - Parcel Activity and Record Management Module
        - Task 5 - Parcel Category Organization Module
- The team may decide how work is divided. The lecturer will not prescribe which member must implement which data structure.
- Each team member is required to use at least **ONE (1)** suitable data structure along with its appropriate algorithms to carry out their assigned task. Although the prototype is developed as a team, assessment is conducted individually.

7. Individual Demonstration

- Demonstrate the functionality implemented by the student.
- Explain the relevant source code.
- Explain the selected data structure and its operations.
- Explain the algorithm(s) used.
- Justify why the selected data structure and algorithm are appropriate.
- Explain at least one technical challenge and how it was resolved.
- Maximum recommended duration: approximately 5 minutes per student. The presentation should focus on the student's individual contribution.
- **PowerPoint** slides are not required.

During the individual evaluation, students may be asked to trace code, explain an operation, discuss time complexity, identify an edge case, explain a design decision, or make a small modification to demonstrate understanding.

8. Submission Requirements

8.1 Group C++ Submission

Submit the zipped folder - complete source code for the integrated system, including:

- .cpp files
- .hpp/header files
- Required .txt, .csv or other data files
- Other supporting files required to compile and run the system

The zip file must adhere to the following name format:

"<GroupNo>_<student ID-leader>_<student ID-member1>_<student ID-member2>.zip"

For example, **"G1_TP012345_TP012344_TP012123.zip"**

8.2 APU - AI Usage Policy form

Each student must submit their own individual APU - AI Usage Policy form (signed). Do not put in zipped file.

8.3 Workload Matrix

Sample:

TP #	Name	Task	Explanation
TP000001	Amy Chan	Parcel Processing	
TP000002	Joshua Ananda	Service Request Management	
TP000003	Brian Nelson	Resource Management	
TP000004	Amina Abu	Activity Management	
TP000005	Divinesh Kumar	System Integration & Testing	

8.4 Student Submission Checklist

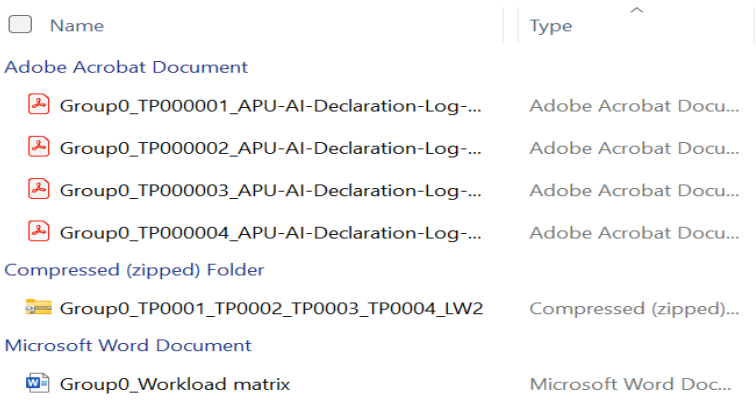
☐ One integrated C++ prototype (submitted to **Moodle**).

☐ APU AI - Usage Policy form for each student (submitted to **Moodle**).

☐ Workload matrix in Ms Word (submitted to **Moodle**).

☐ Face to face presentation session with module lecture (attended and participated - **Face to Face**)

Sample Moodle submission:



9. Assessment Structe

A. Practical Skills - Problem-Solving Skills (15 Marks)

Criterion
Identify and address technical challenges
Use of appropriate data structures and algorithms
Implementation of functional requirements
Code quality: readability, efficiency and correctness
Quality of individual contribution to the integrated system
Innovation and creativity in the solution

B. Q&A with Justification of Data Structures (15 Marks)

Criterion
Data structure selection and justification
Relevance of selected data structures to functionality
Justification of algorithms and operations
Understanding of efficiency, limitations and edge cases
Effectiveness of individual demonstration and responses

10. Academic Integrity and Individual Ownership

- All submitted work must comply with APU academic integrity requirements.
- Each student must understand and be able to explain their own contribution.
- The team must not submit copied or unauthorized source code.
- Students may be asked to explain, trace or demonstrate their code during the individual evaluation.
- An individual evaluation may be used to verify authorship and understanding.